

Assignment: RNN vs LSTM for Fake News Classification

Machine Learning Course

Task Overview

The goal of this assignment is to implement, train, and compare a simple **RNN** and an **LSTM** neural network for text classification using PyTorch. You will work with the Fake/Real News dataset from Kaggle:

[Kaggle: Fake and Real News Dataset](#)

The dataset consists of:

- `Fake.csv` (23,502 fake news articles)
- `True.csv` (21,417 real news articles)

Each file contains the following columns: `Title`, `Text`, `Subject`, `Date`. For this assignment, you may use either `Text` alone or a concatenation of `Title + Text`.

1 Part A: Mathematical Background and Model Architectures

1.1 Problem Definition

We are given a labeled dataset

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N,$$

where each x_i is a token sequence representing a news article, and $y_i \in \{0, 1\}$ is the label (0 = real, 1 = fake).

We train a binary classifier $f_\theta(x)$ using the binary cross-entropy loss:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N [y_i \log \hat{p}_\theta(x_i) + (1 - y_i) \log(1 - \hat{p}_\theta(x_i))],$$

where

$$\hat{p}_\theta(x) = \sigma(z_\theta(x)).$$

1.2 Simple RNN

A classical (Elman-type) RNN computes:

$$\begin{aligned}h_t &= \tanh(W_{ih}x_t + W_{hh}h_{t-1} + b_h), \\o &= W_{ho}h_T + b_o, \quad \hat{p} = \sigma(o).\end{aligned}$$

1.3 LSTM

The LSTM cell is defined as:

$$\begin{aligned}i_t &= \sigma(W_{xi}x_t + W_{hi}h_{t-1} + b_i), \\f_t &= \sigma(W_{xf}x_t + W_{hf}h_{t-1} + b_f), \\o_t &= \sigma(W_{xo}x_t + W_{ho}h_{t-1} + b_o), \\\tilde{c}_t &= \tanh(W_{x\tilde{c}}x_t + W_{h\tilde{c}}h_{t-1} + b_{\tilde{c}}), \\c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \\h_t &= o_t \odot \tanh(c_t).\end{aligned}$$

The output probability is computed in the same way as for the RNN.

2 Part B: Data Preprocessing

2.1 Recommended Steps

- 1) Load both CSV files and assign labels.
- 2) Perform minimal text cleaning (lowercasing, removing excessive punctuation/symbols).
- 3) Tokenize using NLTK/spaCy/TorchText or a custom tokenizer.
- 4) Build a vocabulary of most frequent tokens or use pretrained embeddings.
- 5) Convert tokens to indices and apply padding/truncation to a maximum length L .
- 6) Split into train/validation/test (e.g., 80/10/10).
- 7) Create PyTorch DataLoader objects with batching.

3 Part C: PyTorch Model Skeletons

3.1 Simple RNN Classifier (Skeleton)

```

1 import torch
2 import torch.nn as nn
3
4 class SimpleRNNClassifier(nn.Module):
5     def __init__(
6         self, vocab_size, emb_dim, hidden_dim,
7         num_layers=1, dropout=0.2
8     ):
9         super().__init__()
10        self.embedding = nn.Embedding(
11            vocab_size, emb_dim, padding_idx=0
12        )
13        self.rnn = nn.RNN(
14            input_size=emb_dim,
15            hidden_size=hidden_dim,
16            num_layers=num_layers,
17            batch_first=True,
18            nonlinearity='tanh',
19            dropout=(dropout if num_layers > 1 else 0.0)
20        )
21        self.fc = nn.Linear(hidden_dim, 1)
22
23    def forward(self, x):
24        emb = self.embedding(x)           # (B, L, emb_dim)
25        out, h_n = self.rnn(emb)         # h_n: (layers,
26            B, H)                        # (B, H)
27        last = h_n[-1]
28        logits = self.fc(last).squeeze(1)
29        return torch.sigmoid(logits), logits

```

3.2 LSTM Classifier (Skeleton)

```

1 class LSTMClassifier(nn.Module):
2     def __init__(
3         self, vocab_size, emb_dim, hidden_dim,
4         num_layers=1, bidirectional=False, dropout=0.2
5     ):
6         super().__init__()
7         self.embedding = nn.Embedding(
8             vocab_size, emb_dim, padding_idx=0
9         )
10        self.lstm = nn.LSTM(
11            input_size=emb_dim,
12            hidden_size=hidden_dim,
13            num_layers=num_layers,
14            batch_first=True,
15            bidirectional=bidirectional,

```

```

16         dropout=(dropout if num_layers > 1 else 0.0)
17     )
18     factor = 2 if bidirectional else 1
19     self.fc = nn.Linear(hidden_dim * factor, 1)
20
21     def forward(self, x):
22         emb = self.embedding(x)
23         out, (h_n, c_n) = self.lstm(emb)
24         last = h_n[-1]                                # adjust if
                bidirectional
25         logits = self.fc(last).squeeze(1)
26         return torch.sigmoid(logits), logits

```

3.3 Training Loop (Skeleton)

```

1 criterion = nn.BCEWithLogitsLoss()
2 optimizer = torch.optim.Adam(model.parameters(), lr=LR)
3
4 for epoch in range(num_epochs):
5     model.train()
6     for x_batch, y_batch in train_loader:
7         optimizer.zero_grad()
8         prob, logits = model(x_batch)
9         loss = criterion(logits, y_batch.float())
10        loss.backward()
11        torch.nn.utils.clip_grad_norm_(
12            model.parameters(), 5.0
13        )
14        optimizer.step()
15
16        # Validation:
17        # model.eval()
18        # compute metrics without gradients

```

4 Part D: Metrics and Visualizations

You must evaluate your models using the following metrics:

- Accuracy
- Precision
- Recall
- F1-score
- ROC AUC

Required plots:

1. Training and validation loss curves.
2. Training and validation accuracy or F1 curves.
3. Confusion matrix on the test set.
4. ROC curve with AUC score.

5 Part E: Hyperparameter Search and Model Comparison

5.1 Hyperparameters to Explore

- Embedding size $d \in \{50, 100, 200\}$
- Hidden size $H \in \{64, 128, 256\}$
- Number of layers $L \in \{1, 2\}$
- Bidirectional LSTM: $\{\text{True}, \text{False}\}$
- Learning rate: $\{10^{-3}, 5 \cdot 10^{-4}, 10^{-4}\}$
- Dropout: $\{0.0, 0.2, 0.5\}$
- Batch size: $\{32, 64\}$
- Optional: pretrained embeddings

5.2 Experimental Protocol

1. Fix random seeds for reproducibility.
2. Perform training and validation for each hyperparameter configuration.
3. Repeat key experiments multiple times and report average metrics.
4. Compare RNN vs LSTM in terms of:
 - F1-score (primary metric)
 - ROC AUC
 - Training time per epoch
 - Model size / memory usage

6 Part F: Report Structure

Your final report must include:

1. Introduction and problem description.
2. Dataset overview.
3. RNN and LSTM descriptions with formulas.
4. Data preprocessing pipeline.
5. Experimental setup and hyperparameters.
6. Results with figures and tables.
7. Analysis and comparison of RNN vs LSTM.
8. Conclusions.